

AAMT-WP-12: Filesystem-Mediated Multi-Agent Coordination

AAMT-WP-12 | AsAManThinks Research | May 2026 | DOI:
10.5281/zenodo.19600795

Filesystem-Mediated Multi-Agent Coordination: PID Registries, Cross- Repo Path Aliases, and Idempotent .env Injection as Zero-Protocol Integration Primitives

Weslyn Whitehead Jr.

AsAManThinks Research, Berkeley, CA, USA |
weslyn@asamanthinks.com

ORCID: [to be registered](#) | License: CC BY 4.0

Filesystem-Mediated Multi-Agent Coordination: PID Registries, Cross- Repo Path Aliases, and Idempotent .env Injection as Zero-Protocol Integration Primitives

AsAManThinks Research — Working Paper WP-12

Weslyn Whitehead Jr. (Docwes / Yare Ace NoK)

ORCID: <https://orcid.org/0009-0005-7707-3210>

May 2026 · MaiiaM Alchemist v0.4.1

DOI: 10.5281/zenodo.19600795

Abstract

Modern AI development platforms coordinate heterogeneous agents — language model sidecars, MCP servers, document stores, inference proxies, and UI shells — across process boundaries, language runtimes, and repository roots. We present **Filesystem-Mediated Multi-Agent Coordination (FM-MAC)**, the architectural pattern implemented in MaiiaM Alchemist, which uses five filesystem primitives as the sole coordination substrate: (1) a **PID registry** in `userData/service-pids/` for cross-session process liveness tracking; (2) a `root:` **path alias system** for cross-repository working directory resolution; (3) **idempotent structured .env mutation** for zero-downtime service integration; (4) **filesystem presence probing** for platform capability detection; and (5) **typed artifact files** (`deploy-spec.json`, `expert-manifest.json`, `lineage.json`) as durable, cross-language, asynchronous coordination contracts. We demonstrate that for the class of coordination tasks that are not latency-sensitive, filesystem primitives outperform direct RPC on resilience,

debuggability, and language-portability dimensions. Direct RPC (stdio JSON-RPC) is reserved exclusively for latency-sensitive real-time operations (streaming inference, training step events). The full implementation is present in MaiaM Alchemist v0.4.1 and is verified against a 20-suite eval harness.

1. Motivation

Multi-agent AI systems are typically designed around one of two coordination models: **message-passing** (agents communicate via queues, buses, or RPC) or **shared memory** (agents read and write a common data store). Both have well-known tradeoffs. Message-passing requires both ends to be running simultaneously; shared memory requires lock management and consistent schema.

A third coordination model is underexplored in the AI systems literature: **filesystem-as-coordination-substrate**. In this model, agents coordinate by reading and writing well-known file paths, without requiring simultaneous execution, language-compatible runtimes, or protocol negotiation. This model is the default in Unix toolchains (Makefile dependency tracking, lock files, PID files) but has not been systematically applied to AI development platform coordination.

MaiaM Alchemist implements FM-MAC across five distinct mechanisms, each addressing a specific coordination problem that message-passing or shared-memory approaches would solve less elegantly.

2. Mechanism 1: PID Registry for Cross-Session Liveness

2.1 Problem

The Alchemist desktop orchestrates up to 12 heterogeneous services across three groups (`infra`, `aamt`, `ui`). Services may have been started by a previous session that crashed or was force-quit. On startup, the new session needs to determine which services are already running without starting a separate health-check server that would itself be a service to manage.

2.2 Implementation

The `ServiceManager` class maintains a PID registry at:

```
~/Library/Application Support/MaiaM Alchemist/service-pids/<serviceId>
```

Each file contains a single ASCII integer: the PID of the running service process. On service start:

```
writePidFile(serviceId, pid) {
    writeFileSync(this.pidFilePath(serviceId), String(pid));
}
```

On liveness check:

```
readPidFile(serviceId) {
  const pid = parseInt(readFileSync(filePath, 'utf8').trim(), 10);
  try {
    process.kill(pid, 0); // signal 0: liveness probe, no effect
    return pid;
  } catch {
    return null; // ESRCH: process not found; file is stale
  }
}
```

`process.kill(pid, 0)` is a Unix idiom: it sends no signal but returns an error if the PID does not exist. This gives an $O(1)$ liveness check without any IPC.

2.3 Stale port cleanup

When a service's PID file is stale (process died without cleanup), the port may still be bound by a zombie. The `cleanupStalePort` method resolves this:

```
cleanupStalePort(port) {
  const output = execSync(`lsof -i :${port} -sTCP:LISTEN -t`, {
    timeout: 3000});
  for (const pid of output.split('\n').map(Number).filter(Boolean)) {
    process.kill(pid, 'SIGKILL');
  }
}
```

This runs before every `start()` call, ensuring port availability without requiring a service registry daemon.

2.4 Properties

The PID registry is resilient to host crashes (files persist), language-agnostic (any language can read an ASCII integer file), and requires no daemon. Its failure modes are all visible: a stale file with a live PID signals a port conflict; a stale file with a dead PID is transparently ignored.

3. Mechanism 2: `root:` Path Aliases for Cross-Repository Resolution

2.1 Problem

Alchemist coordinates services across two separate repositories at different filesystem locations (`/Volumes/Abundance/asamanthinks-platform` and `/Volumes/Abundance/maiam-alchemist`). Service working directories must be resolved at runtime relative to repository roots that differ per developer machine. Hardcoded absolute paths would break on any machine that isn't the primary developer's.

3.2 Implementation

Service definitions use a `root:` prefix that encodes repository identity:

```
// Services in the AAMT monorepo:
{ cwd: "root:aamt/packages/wings-mcp-server", ... }
{ cwd: "root:aamt/aamt-mcp", ... }

// Services in the MaiaM Platform repo:
{ cwd: "root:platform/MaiaM Integration", ... }
```

The `ServiceManager.resolveCwd()` method resolves these at runtime:

```
resolveCwd(cwd) {
  if (cwd.startsWith('root:aamt/')) {
    return path.resolve(app.getAppPath(), '../../../',
      cwd.replace('root:aamt/', ''));
  }
  if (cwd.startsWith('root:platform/')) {
    return path.resolve(app.getAppPath(), '../../../../MaiaM
Platform',
      cwd.replace('root:platform/', ''));
  }
  return cwd; // absolute path fallback
}
```

3.3 Platform mode detection

A related mechanism uses filesystem presence as a capability probe:

```
function detectPlatformMode() {
  if (existsSync('/Volumes/Abundance/asamanthinks-platform')) r
return 'premium';
  return 'base';
}
```

No network call, no configuration file, no build flag. The presence of the platform repository on the expected volume is the signal. This is portable across machines (any machine with the volume mounted gets `premium` mode), deterministic, and instantly readable without IPC.

3.4 Properties

The alias system makes service definitions declarative and location-independent while keeping the resolution logic in a single function. Adding a new repository root requires one line in `resolveCwd`, not a configuration file change distributed across all service definitions.

4. Mechanism 3: Idempotent Structured .env Mutation

4.1 Problem

AnythingLLM (a third-party document-chat application) needs to be configured to route LLM requests to Alchemist's OpenAI-compatible endpoint rather than to OpenAI. AnythingLLM reads its configuration from `.env` files at startup. It is not Alchemist-aware and provides no integration API.

4.2 Implementation

The `AnythingLLMBridge` class integrates Alchemist with AnythingLLM via **six targeted .env mutations**, leaving all other configuration untouched:

```
const BRIDGE_KEYS = new Set([
  'LLM_PROVIDER',
  'GENERIC_OPEN_AI_BASE_PATH',
  'GENERIC_OPEN_AI_MODEL_PREF',
  'GENERIC_OPEN_AI_API_KEY',
  'GENERIC_OPEN_AI_MAX_TOKENS',
  'GENERIC_OPEN_AI_CUSTOM_HEADERS'
]);

async configure(config) {
  const bridgeValues = {
    LLM_PROVIDER: 'generic-openai',
    GENERIC_OPEN_AI_BASE_PATH: config.alchemistBaseUrl, // http://localhost:9202/v1
    GENERIC_OPEN_AI_MODEL_PREF: config.modelPref,
    GENERIC_OPEN_AI_API_KEY: 'maaiam-unsigned',
    GENERIC_OPEN_AI_MAX_TOKENS: String(config.maxTokens),
    GENERIC_OPEN_AI_CUSTOM_HEADERS: '{"X-AAMT-Unsigned":"1"}'
  };
  mergeAndWrite(forkEnv, bridgeValues); // fork/server/.env
  mergeAndWrite(infraEnv, bridgeValues); // infra/.env
}
```

The `mergeAndWrite` function reads the existing file, updates only the six bridge keys line by line, and appends any missing keys. Existing configuration is preserved exactly. `unconfigure()` performs the inverse: removes only the six bridge keys:

```
async unconfigure() {
  const filtered = lines.filter(line => !BRIDGE_KEYS.has(parseKey(line)));
  writeFileSync(filePath, filtered.join('\n'));
}
```

4.3 Properties

This integration is: - **Idempotent:** calling `configure()` twice produces the same result as once - **Reversible:** `unconfigure()` restores the file to its pre-integration state exactly - **Zero-downtime:** AnythingLLM reads these values on its next restart; no restart of Alchemist required - **Non-invasive:** no fork of AnythingLLM, no plugin API, no webhook — the integration handshake is a file write

The pattern generalizes to any application that reads configuration from `.env` files. The `BRIDGE_KEYS` set is the declarative contract between integrator and target.

5. Mechanism 4: Typed Artifact Files as Asynchronous Coordination Contracts

5.1 Problem

Training produces a model. Deployment consumes that model. These events may be separated by hours, days, or months. The training process (Python) and the deployment consumers (TypeScript CLI, PHP plugin, C# Unity bridge) speak different languages and run at different times. They need a shared coordination contract.

5.2 The artifact contract

The run directory `runs/<run_id>/` contains a set of typed artifact files that serve as the coordination contract:

```

runs/<run_id>/
├─ deploy-spec.json           # canonical deploy descriptor
├─ expert-manifest.json      # adapter → shell → expert mapping
├─ lineage.json              # tar of relevant lineage nodes
└─ sha256.txt                 # integrity verification

```

`deploy-spec.json` is the universal adapter. The TypeScript `unity-bridge` CLI reads it. The PHP WordPress plugin reads it. The mobile GGUF quantizer reads it. None of these consumers know about each other. They each read the same file independently, at their own time, from their own language runtime.

5.3 Path-as-identity

The `run_id` (a UUIDv7) serves as the coordination token. Any consumer that knows the `run_id` can derive the full artifact tree by path construction:

```

runs/<run_id>/deploy-spec.json
runs/<run_id>/adapters/unity_csharp.safetensors
runs/<run_id>/model.onnx

```

No registry lookup, no service discovery, no API call. The filesystem is the registry.

5.4 Properties

Artifact files are durable (survive process crashes), inspectable (JSON, readable by any tool), versioned (the file schema version is included in the JSON), and language-portable. The append-only `lineage.db` extends this pattern with temporal ordering and tamper detection.

6. Mechanism 5: Ordered Group Startup with Filesystem-Gated Preconditions

6.1 Problem

The AAMT service group has a hard ordering constraint: MCP servers must be fully started before AnythingLLM, because AnythingLLM dials all registered MCP servers at boot. A race condition here silently disables MCP tools for the session.

6.2 Implementation

`startGroup` enforces order via sequential execution with a fixed inter-service delay:

```

async startGroup(group) {
  const services = SERVICES.filter(s => s.group === group && !s
.optional);
  for (const svc of services) {
    await this.start(svc.id);
    await this.delay(800); // 800ms: enough for bind() to s
ucceed
  }
}

```

The 800ms delay is a filesystem-observable precondition: within 800ms, a Node.js service will have opened its TCP port (observable via `lsof`). This is a filesystem-mediated readiness signal, not a health-check endpoint.

The `conflicts` field on service definitions uses the same principle:

```
{ id: 'anythingllm-infra', conflicts: ['anythingllm'], ... }
```

Before starting a service, the manager checks if any conflicting service is running (via PID file + `kill -0`). The conflict resolution is a filesystem read, not a runtime API call.

7. Comparative Analysis

Coordination Task	FM-MAC Mechanism	Alternative	FM-MAC Advantage
Cross-session liveness	PID file + <code>kill -0</code>	Health-check HTTP server	No daemon required; works after crash
Cross-repo paths	<code>root: alias + existsSync</code>	Config file / env var	Declarative location-transparency
Third-party integration	<code>.env</code> mutation	Plugin API / webhook	Works with any <code>.env</code> -reading service
Platform capability	<code>existsSync('/Volumes/...')</code>	Feature flag service	Zero latency; no service dependency
Cross-language contracts	Typed JSON artifact files	gRPC / REST API	Language-agnostic; async-safe
Service startup ordering	Sequential <code>delay()</code>	Health-check polling	Simpler; sufficient for bind latency

The pattern is not universally applicable. FM-MAC does not work for: latency-sensitive operations (streaming inference uses stdio JSON-RPC), real-time state push (metrics use the streaming sidecar protocol), or large payload transfers (model weights use direct file I/O outside the coordination layer).

8. Conclusion

FM-MAC demonstrates that for the specific class of coordination tasks that are asynchronous, cross-language, and resilience-critical, filesystem primitives provide a simpler, more debuggable, and more portable coordination substrate than message-passing or shared-memory alternatives. The five mechanisms — PID registry, path aliases, idempotent `.env` mutation, typed artifact files, and filesystem-gated startup ordering — together cover the full coordination surface of a production AI development platform with no coordination

daemon, no service registry, and no protocol negotiation overhead. The full implementation is open and inspectable in MaiiaM Alchemist v0.4.1.

References

- Tanenbaum, A., Van Steen, M. (2007). Distributed Systems: Principles and Paradigms.
- Raymond, E.S. (2003). The Art of Unix Programming. \$Rule of Separation, \$Rule of Representation.
- Bernstein, P.A., Newcomer, E. (2009). Principles of Transaction Processing. \$Idempotency.
- Whitehead, W. (2026). MaiiaM Wings: Consciousness-Aligned Language Architecture. Zenodo. DOI: 10.5281/zenodo.19600795